Project 1
CSC 4444 Dong Lao
Guido M Fajardo Gabrie
October 20, 2026

Classifying Digits

Abstract

I built and compared five classifiers for handwritten digits on the MNIST dataset. K Nearest Neighbors and Naive Bayes in NumPy and a linear classifier, a multilayer perceptron (MLP) and a convolutional neural network (CNN) in PyTorch. All of these were trained in the 48,000 images and tested on the same 12,000. The CNN was the most accurate with a 98.70%, followed by the MLP with a 97.69%, KNN with 97.21, the linear classifier with 84.54% and Naive Bayes with 83.93%. I also ran experiment on hyperparameters and loss functions. The choice of the loss function mostly changed accuracy more than the model size did. Replacing L2 loss function with cross entropy raised the linear classifier from 85% to 91%, and an MLP trained with L2 loss did worse than a linear model trained with cross entropy. But across almost every method one of the most common mistakes was reading a 4 as a 9.

Description

The Goal of this project is to look at a 28x28 grayscale image of a handwritten digit and recognize or decide which digit from 0 to 9, it shows. This is a classification problem the input would be the image, and the output is one of the ten classes (Lao, “Classifiers”). It cannot be solved with fixed rules, because everyone writes different. A 4 can be open or closed at the top and an 7 can have a line through it or not. The classifier has to learn the patterns from labeled examples

Digit recognition is used to read postal codes, bank cheques and handwritten forms. MNIST Is also a standard benchmark in machine learning (Lao, “Classifiers”). It is small enough to train it in your laptop but hard enough that different methods can be used and give different results. That makes it a good way to compare how classical methods and neural networks handle the same problem.

AI Techniques

K Nearest Neighbors (KNN). KNN has no training step. It stores the training set, and to classify a new image it finds the k closest training images and lets them vote (Lao, “Classifiers”). I used Euclidean distance on the 784-pixel values. Squaring the distance doesn’t change which neighbors are closest it just adds more processing time to it, so I decided not to square root them. To make it fast, I wrote the squared distance as $|a-b|^2 = |a|^2 + |b|^2 – 2a*b$. The $a*b$ term for all pairs of images becomes a single matrix multiplication. And I processed the validation images in batches of 500 to keep memory use small, since if all of them were processed at once it will be around 1.8 Tb of memory. When the votes tie, the digit of the nearest neighbor wins.

Naive Bayes. Each pixel is binarized 1 if its value is above 0.5 and 0 if its otherwise. For each digit, I estimate the probability that each pixel is on. A new image is then classified with Bayes’ rule, assuming that the pixels are independent of each other given the digit (Lao, “Classifiers”). Multiplying 784 small probabilities underflows to zero on a

computer, so I added log probabilities instead. I also used Laplace smoothing (Lao, “Classifiers”), (count + 1) / (N + 2), so that no probability is exactly 1 or 0.

Linear Classifier. This model computes ten scores, y = Wx, one per digit, and it then predicts the digit with the highest score (Lao, “Classifiers”) W has shape 10 x 784. The labels are turned into one hot vector, and W is trained to minimize the L2 (squared) loss between the scores and those vectors (Lao, “Regression”). Training uses gradient descent: compute the loss, compute its gradient with respect to W and then step against the gradient by a learning rate (Lao, “Optimization”; 3Blue1Brown, “Gradient Descent”). I used PyTorch’s nn.linear with no bias, to basically match y = Wx and mini batch stochastic gradient descent (SGD).

Multilayer Perceptron (MLP). The MLP stack fully connected layers, 784 → 256 → 128 →10, with a ReLU activation after each hidden layer (Lao, “Neural Networks”; Lao, “Neural Networks II”). Without the ReLU, the stacked layers would collapse into one linear layer. With it, the network can combine simple features into more complex ones (3Blue1Brown, “Neural Network”). It is trained with SGD (Lao, “Optimization”; Lao, “Neural Networks II”) and cross entropy loss. In PyTorch, CrossEntropyLosss expects raw scores and applies SoftMax internally (PyTorch; Lao, “Regression”), so the network has no SoftMax layer at the end.

Convolutional neural networks (CNN). The CNN keeps the image in 2D. It has two block of Conv(3x3) →ReLU → MaxPool(2x2), with 32 and then 64 filters, followed by a linear lyer to 10 outputs (Lao, “Neural Networks II”; Lao, “Computer Vision”). The output size of each layer follows [(n – k + 2p)/ s] +1 so the image shrinks 28 → 26 → 13 → 11 → 5, and the final layer receives 64 x 5 x 5 = 1,600 values. Because each filter is reused at every position, the same edge detector works anywhere in the image. Pooling also makes the network tolerant of small shifts (Lao, “Neural Networks II”; Lao, “Computer Vision”).

Dataset

I used the MNIST images provided on Moodle, 60,000 PNG files, each 28x28 grayscale, stored in one folder per digit. Each digit has between 5,421 and 6,742 images, and the folder name gives the label. I loaded the files in sorted order, scaled the pixels from 0 to 255 to [0,1], after that shuffled them with a fixed seed (42) and split them 80/20 into 48,000 training and 12,000 validation images (Lao, “Classifiers”). Every method uses exactly the same split, so the results are directly comparable. The PyTorch models received pixels rescaled to [-1,1]. Images were flattened to 784 values for KNN, Naive Bayes, the linear classifier and the MLP and kept as 1 x 28 x 28 for the CNN

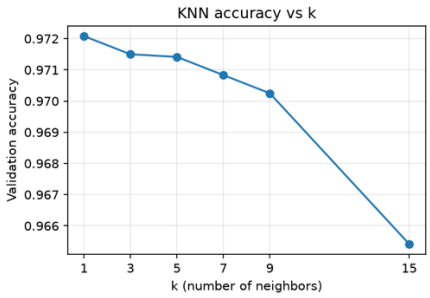
Implementation Tools

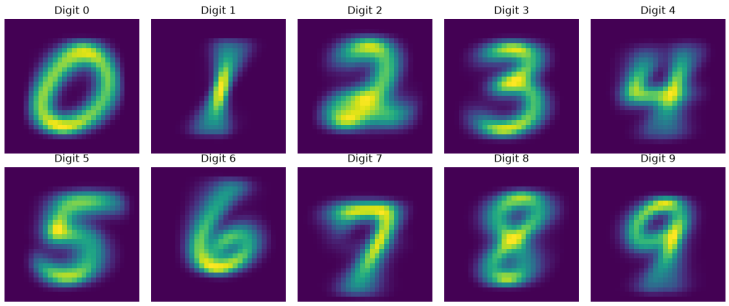
The project is written in Python 3.12.13. KNN, Naive Bayes and the data loading use NumPy (NumPy Developers). The linear classifier, the MLP and the CNN use PyTorch (PyTorch). Pillow reads the PNG files, Matplotlib makes the figures, and pytest runs 34 unit tests that check each function on small handmade examples. Training ran on Apple M series laptop, using its GPU through PyTorch’s MPS backend. Each method has its own file, plus shared files for loading the data and building the PyTorch data loaders.

Results and Analysis

Method	Accuracy	Errors (of 12,000)	Notes
CNN	98.70%	156	34,826 parameters, 10 epochs, about 99 s
MLP	97.69%	277	235,146 parameters, 15 epochs, 27 s
KNN (k = 1)	97.21%	335	stores all 48,000 images, 66 s to predict
Linear (L2)	84.54%	1,855	7,840 weights, 20 epochs, 28 s
Naive Bayes	83.93%	1,928	about 1.6 s to train and predict

KNN. Accuracy fell slowly as k increased, from 97.21% at k = 1 to 96.54% at k = 15. The gap between k = 1 and k = 3 was only 7 images, so it isn't a meaningful difference with. With 48,000 training images the nearest neighbor is usually already very similar, so extra voters mostly add noise.



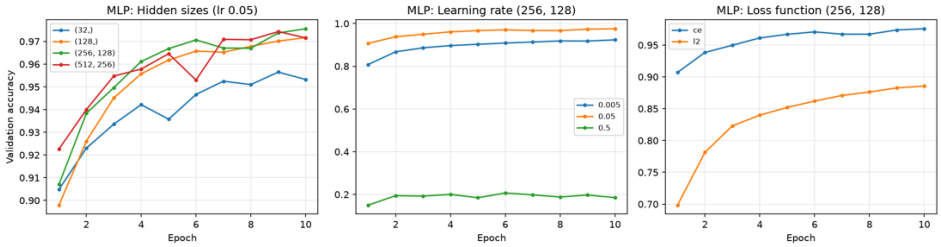


Naive Bayes. The probability maps look like blurry version of each digit. The independence assumption is clearly false, since neighboring pixels in a stroke turn on together, and binarizing throws away the grey edges of the strokes. This makes Naive Bayes the least accurate method but it's the smallest and fastest of them all. Smoothing barely mattered: alpha from 0.001 t 1 all gave about 84.0% and only one alpha = 100 hurt (82.4%). With about 4,800 images per digit, adding 1 to a count changes almost nothing. When I repeated it on five different random splits, accuracy ranged from 83.0% to 83.9% (mean 83.5%). So, differences smaller than about 0.3 points are just noise, and the seed 42 split happened to be the best of five

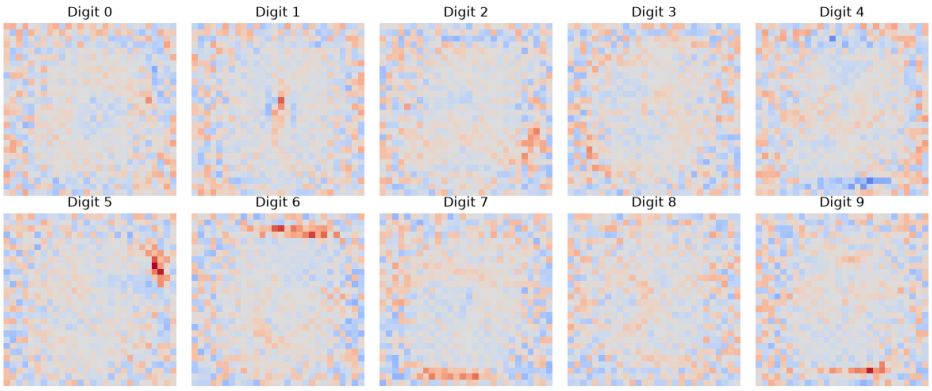
Linear Classifier. My first run used a learning rate of 0.05, and the loss became NaN on the first epoch. With pixels in [-1,1] the background is -1 instead of 0, so every input vector is large, and each gradient step just overshot until the weights blew up (Lao, "Optimization"). A learning rate of 0.01 fixed it. Plotting each row of W as 28 x 28 image shows a rough template for each digit. The confusion matrix showed the model predicting "1" far too often, 1,722 predictions for only 1,347 that were real ones, mostly coming from 8s and 2s

Loss Function Experiment. The biggest finding came from changing only the loss function. With cross entropy instead of L2 (Claude recommended this one), the same linear model went from 85.3% to 91.1% after 10 epochs. L2 loss also lowered the MLP from 97.6% to

88.6%, and the CNN from 98.4% to 93.7% after 5 epochs. L2 tries to make every score exactly 0 or 1 (Lao, “Regression”), so it keeps spending effort on images that are already classified correctly. Cross entropy only cares that the correct digit has the highest score (PyTorch).



MLP and CNN. The MLP passed 96% by epoch 4. After epoch 10, its training loss kept falling while validation accuracy stayed flat around 97.7%, which is the start of overfitting (Lao, “Classifiers”; Lao, “Regression”). A larger MLP with (512, 256) hidden units wasn’t more accurate than (256, 128) after 10 epochs, even with twice the parameters. The learning rate had to be in the right range: 0.005 was too slow (92.4%), and 0.5 diverged (18.5%) (Lao, “Optimization”). The CNN was the best model overall, with 7 times fewer parameters than the MLP. Even a CNN with only 8 and 16 filters (5,258 parameters) reached 98.1%, more than the MLP. Many of the first-layer filters look like edge detectors at different angles (Lao, “Computer Vision”).



Failure modes. The confusion matrices (Lao, “Classifiers”) show the same weakness in four of the five methods: a 4 classified as a 9. It happened 178 times for Naive Bayes, 35 for KNN, 25 for the MLP and 13 for the CNN. Other frequent confusions were between 3, 5 and 8, and between 7 and 9, all digits whose strokes mostly overlap. Many of the misclassified images are badly written, and a person would struggle to read them too.

6. References and AI Assistance

3Blue1Brown. “But What Is a Neural Network? | Deep Learning Chapter 1.” *YouTube*, 2017, www.youtube.com/watch?v=aircAruvnKk.

---. “Gradient Descent, How Neural Networks Learn | Deep Learning Chapter 2.” *YouTube*, 2017, www.youtube.com/watch?v=IHZwWFHWa-w.

Lao, Dong. “Classifiers.” *CSC 4444: Artificial Intelligence*, Lecture 8, Louisiana State University, Fall 2026. Lecture slides.

---. “Computer Vision.” *CSC 4444: Artificial Intelligence*, Lecture 12, Louisiana State University, Fall 2026. Lecture slides.

---. “Neural Networks.” *CSC 4444: Artificial Intelligence*, Lecture 10, Louisiana State University, Fall 2026. Lecture slides.

---. “Neural Networks II.” *CSC 4444: Artificial Intelligence*, Lecture 11, Louisiana State University, Fall 2026. Lecture slides.

---. “Optimization.” *CSC 4444: Artificial Intelligence*, Lecture 5, Louisiana State University, Fall 2026. Lecture slides.

---. “Regression.” *CSC 4444: Artificial Intelligence*, Lecture 9, Louisiana State University, Fall 2026. Lecture slides.

NumPy Developers. *NumPy Documentation*. numpy.org/doc/stable/. Accessed 20 Oct. 2026.

PyTorch. “torch.nn.CrossEntropyLoss.” *PyTorch 2.14 Documentation*, docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html. Accessed 20 Oct. 2026.

AI Use Statement. I used Claude (Anthropic), through Claude Code, as a tutor and assistant during this project.

- **Setup and structure:** it set up my Python environment and created skeleton files listing each function’s inputs, outputs and steps.
- **Explanations:** it explained the concepts I didn’t know, including vectorized distances, gradient descent, Laplace smoothing, the PyTorch training loop and the conv-layer size formula. It quizzed me on them before I wrote code.
- **My code:** I wrote the data loader, the KNN and Naive Bayes functions, the linear, MLP and CNN models, their training loops, and most of the unit tests.
- **Code reviews:** Claude reviewed my code and pointed out bugs, which I either fixed or asked it to fix. Examples: unsorted file loading, a return inside a loop, importing the wrong predict, and a learning rate that caused NaN loss.
- **Code Claude wrote:** the GPU/CPU device selection, the KNN analysis and plotting section, the confusion-matrix function, the experiment script, and the experiment log.

All numbers in this report come from running the code myself.